assignment 3 - NLPROC-ALV-B

Anja Kieburg, Friedemann Decker

Task 2 A

Korpus TIGER 2.2.doc (April 2018) from: https://www.ims.uni-stuttgart.de/en/research/resources/corpora/tiger/

TIGER Corpus

The TIGER Corpus consists of approximate 900,000 tokens (50,000 sentences) of German newspaper text, taken from the Frankfurter Rundschau. The corpus was semi-automatically POS-tagged and annotated with syntactic structure. Moreover, it contains morphological and lemma information for terminal nodes

In [9]:
```
import pandas as pd
```

Step 1: Import TIGER XML into Pandas, using only the entries under `s=Sätze`

In [64]:
```
tiger_corpus = pd.read_xml("tiger_release_aug07.corrected.16012013.xml",
                           xpath="//s/graph/terminals/t",
                           elems_only=False,
                           encoding="ISO-8859-1")
```

In [65]:
```
tiger_corpus.head()
```

Out[65]:

	id	word	lemma	pos	morph	case	number	gender	person	degree	tense	mood	secedge
0	s1_1	``	--	$(	--	--	--	--	--	--	--	--	NaN
1	s1_2	Ross	Ross	NE	Nom.Sg.Masc	Nom	Sg	Masc	--	--	--	--	NaN
2	s1_3	Perot	Perot	NE	Nom.Sg.Masc	Nom	Sg	Masc	--	--	--	--	NaN
3	s1_4	wäre	sein	VAFIN	3.Sg.Past.Subj	--	Sg	--	3	--	Past	Subj	NaN
4	s1_5	vielleicht	vielleicht	ADV	--	--	--	--	--	--	--	--	NaN

Step 2: Required fields: Import "word" and "pos" into the DataFrame

In [66]:
```
words = tiger_corpus[["word", "pos"]]
words.head()
```

Out[66]:

	word	pos
0	``	$(
1	Ross	NE
2	Perot	NE
3	wäre	VAFIN
4	vielleicht	ADV

Step 3: Count the number of unique combinations of "word" AND "pos" = frequency of occurrence in the corpus

In [67]:
```
word_pos_freq = words[["word", "pos"]].value_counts()
words = word_pos_freq.reset_index(name="pos_freq")
words.head()
```

Out[67]:

	word	pos	pos_freq
0	,	$,	43656
1	.	$.	41057
2	der	ART	25286
3	die	ART	21209
4	und	KON	16281

Step 4: Convert "word" to lowercase so that different parts of speech (POS) can be grouped into a "word_lower" category

In [68]:
```
word_lower = words["word"].str.lower()
words.insert(1, "word_lower", word_lower)
words.head()
```

Out[68]:

	word	word_lower	pos	pos_freq
0	,	,	$,	43656
1	.	.	$.	41057
2	der	der	ART	25286
3	die	die	ART	21209
4	und	und	KON	16281

Step 5: Determine the number of words per "word_lower" category and enter it as "word_freq" in each row to calculate p(t|w).

In [69]:
```
word_freq = words.groupby("word_lower")["pos_freq"].transform("sum")
words.insert(4, "word_freq", word_freq)
words.head()
```

Out[69]:

	word	word_lower	pos	pos_freq	word_freq
0	,	,	$,	43656	43659
1	.	.	$.	41057	41057
2	der	der	ART	25286	29662
3	die	die	ART	21209	30024
4	und	und	KON	16281	16786

Step 6: Calculate p_pos = p(t|w)

In [70]:
```
p_pos = words["pos_freq"] / words["word_freq"]
words.insert(5, "p_pos", p_pos)
words.head()
```

Out[70]:

	word	word_lower	pos	pos_freq	word_freq	p_pos
0	,	,	$,	43656	43659	0.999931
1	.	.	$.	41057	41057	1.000000
2	der	der	ART	25286	29662	0.852471
3	die	die	ART	21209	30024	0.706402
4	und	und	KON	16281	16786	0.969915

Step 7: Keep only p_pos_max values in the DataFrame

In [71]:
```
words["max_p_pos"] = words.groupby("word_lower")["p_pos"].transform("max")
words = words[words["p_pos"] == words["max_p_pos"]]
```

```
words.head()
```

Out[71]:

	word	word_lower	pos	pos_freq	word_freq	p_pos	max_p_pos
0	,	,	$,	43656	43659	0.999931	0.999931
1	.	.	$.	41057	41057	1.000000	1.000000
2	der	der	ART	25286	29662	0.852471	0.852471
3	die	die	ART	21209	30024	0.706402	0.706402
4	und	und	KON	16281	16786	0.969915	0.969915

Step 8: Delete columns that aren't needed (clean up). Rename the columns according to the assignment.

In [72]:
```
words = words.drop(columns = ["word_lower", "p_pos"])
```

In [78]:
```
words = words.rename(columns={'word': 'w', 'pos': 't', 'max_p_pos': 'p'})
```

Result: Computation of the tag **t** for each word **w** that maximizes **p(t|w)**. List of the first 30 values:

In [79]:
```
words.head(30)
```

Out[79]:

	w	t	pos_freq	word_freq	p
0	,	$,	43656	43659	0.999931
1	.	$.	41057	41057	1.000000
2	der	ART	25286	29662	0.852471
3	die	ART	21209	30024	0.706402
4	und	KON	16281	16786	0.969915
5	in	APPR	13711	15102	0.907893
6	den	ART	9486	9955	0.952888
7	''	$(	8811	8811	1.000000
8	``	$(	8809	8809	1.000000
9	von	APPR	8182	9033	0.905790
10	mit	APPR	6035	6872	0.878201
11	des	ART	5903	5920	0.997128
12	für	APPR	5859	6328	0.925885
13	sich	PRF	5585	5595	0.998213
14	auf	APPR	5577	6301	0.885098
16	nicht	PTKNEG	5143	5294	0.971477
17	im	APPRART	5137	5899	0.870826
18	das	ART	4742	7977	0.594459
19	dem	ART	4492	5070	0.885996
20	ist	VAFIN	4420	4464	0.990143
21	zu	PTKZU	4402	7333	0.600300
22	eine	ART	4179	4763	0.877388
23	)	$(	4177	4177	1.000000
24	(	$(	4159	4159	1.000000
25	ein	ART	3904	4995	0.781582
26	auch	ADV	3444	4068	0.846608
27	es	PPER	3105	3902	0.795746
29	daß	KOUS	3054	3257	0.937673
30	:	$.	2962	3051	0.970829
31	an	APPR	2900	3852	0.752856

Appendix: Meaning of the Abbreviations Used for Parts of Speech (POS)

In [80]:
```
tiger_corpus_pos_abr = pd.read_xml("tiger_release_aug07.corrected.16012013.xml",
                  xpath="//head/annotation/feature/value",
                  elems_only=False,
                  encoding="ISO-8859-1")
```

In [81]:
```
tiger_corpus_pos_abr.head(56)
```

Out[81]:

	name	value
0	$(	Sonstige Satzzeichen; satzintern
1	$,	Komma
2	$.	Satzbeendende Interpunktion
3	--	<Nicht zugeordnet>
4	ADJA	Attributives Adjektiv
5	ADJD	Adverbiales oder prädikatives Adjektiv
6	ADV	Adverb
7	APPO	Postposition
8	APPR	Präposition; Zirkumposition links
9	APPRART	Präposition mit Artikel
10	APZR	Zirkumposition rechts
11	ART	Bestimmter oder unbestimmter Artikel
12	CARD	Kardinalzahl
13	FM	Fremdsprachliches Material
14	ITJ	Interjektion
15	KOKOM	Vergleichspartikel, ohne Satz
16	KON	Nebenordnende Konjunktion
17	KOUI	Unterordnende Konjunktion mit zu und Infinitiv
18	KOUS	Unterordnende Konjunktion mit Satz
19	NE	Eigennamen
20	NN	Normales Nomen
21	NNE	Kombination aus Nomen und Eigenname
22	PDAT	Attribuierendes Demonstrativpronomen
23	PDS	Substituierendes Demonstrativpronomen
24	PIAT	Attribuierendes Indefinitpronomen
25	PIS	Substituierendes Indefinitpronomen
26	PPER	Ersetzbares Personalpronomen
27	PPOSAT	Attribuierendes Possessivpronomen
28	PPOSS	Substituierendes Possessivpronomen
29	PRELAT	Attribuierendes Relativpronomen
30	PRELS	Substituierendes Relativpronomen
31	PRF	Reflexives Personalpronomen
32	PROAV	Pronominaladverb
33	PTKA	Partikel bei Adjektiv oder Adverb
34	PTKANT	Antwortpartikel
35	PTKNEG	Negationspartikel
36	PTKVZ	Abgetrennter Verbzusatz
37	PTKZU	zu vor Infinitiv
38	PWAT	Attribuierendes Interrogativpronomen
39	PWAV	Adverbiales Interrogativ- oder Relativpronomen
40	PWS	Substituierendes Interrogativpronomen

	name	value
41	TRUNC	Kompositions-Erstglied
42	UNKNOWN	Unbekanntes Tag aus Einlesen aus Korpusdatei
43	VAFIN	Finites Verb, aux
44	VAIMP	Imperativ, aux
45	VAINF	Infinitiv, aux
46	VAPP	Partizip Perfekt, aux
47	VMFIN	Finites Verb, modal
48	VMINF	Infinitiv, modal
49	VMPP	Partizip Perfekt, modal
50	VVFIN	Finites Verb, voll
51	VVIMP	Imperativ, voll
52	VVINF	Infinitiv, voll
53	VVIZU	Infinitiv mit zu, voll
54	VVPP	Partizip Perfekt, voll
55	XY	Nichtwort, Sonderzeichen

In [ ]: